

AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD
Problem Solving Using C (BMC-102)
Important Questions with Solution

Unit 1

1. Define the algorithm and explain the characteristics of an Algorithm. Write an algorithm to find the roots of a quadratic equation.

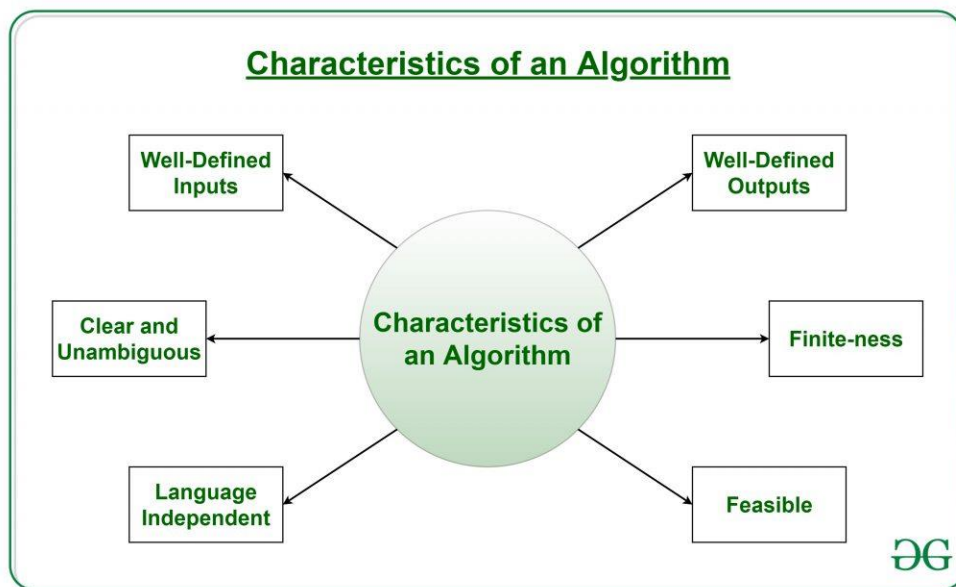
Ans. The word **Algorithm** means " A set of finite rules or instructions to be followed in calculations or other problem-solving operations "

Or

" A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations".

Why it is important?

1. Algorithms are necessary for **solving complex problems** efficiently and effectively.
2. They help to **automate processes** and make them more **reliable, faster, and easier to perform**.
3. Algorithms also enable computers to perform tasks that would be difficult or impossible for humans to do manually.
4. They are **used in various fields such as mathematics, computer science, engineering, finance**, and many others to optimize processes, analyze data, make predictions, and provide solutions to problems.



Characteristics of an algorithm:

- **Clear and Unambiguous:** The algorithm should be unambiguous i.e each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs:** If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Input:** An algorithm has zero or more inputs.
- **Output:** An algorithm produces at least one output.
- **Definiteness:** an algorithm must be unambiguous, precise, and easy to interpret..
- **Finiteness:** Finiteness means Algorithm should be terminated after a finite number of steps. Also, each step should be finished in a finite amount of time.
- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil

An algorithm to find the roots of a quadratic equation:

Step 1: Start

Step 2: Read a, b, c values

Step 3: Compute $d = b^2 - (4*a*c)$

Step 4: if $d > 0$ then

$r1 = b + \sqrt{d}/(2*a)$

$r2 = b - \sqrt{d}/(2*a)$

Step 5: Otherwise if $d = 0$ then

compute $r1 = -b/2a$, $r2 = -b/2a$

Step 6: print r1, r2 values

Step 7: Otherwise if $d < 0$ then print roots are imaginary

Step 8: Stop

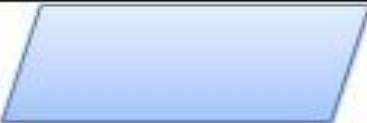
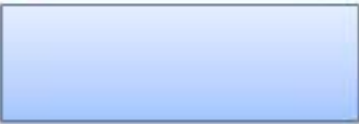
2. What is the use of flowcharts? Explain the symbols used in the flowchart. Draw a flowchart to find the maximum among three numbers inputted by the user.

Ans: Flowcharts are nothing but the graphical representation of the data or the algorithm for a better understanding of the code visually.

- It displays step-by-step solutions to a problem, algorithm, or process.
- It is a pictorial way of representing steps that are preferred by most beginner-level programmers

- A flowchart is a picture of boxes that indicates the process flow sequentially.
- Since a flowchart is a pictorial representation of a process or algorithm, it's easy to interpret and understand the process.
- To draw a flowchart, certain rules need to be followed which are followed by all professionals to draw a flowchart and are widely accepted all over the countries

Flowchart Symbols:

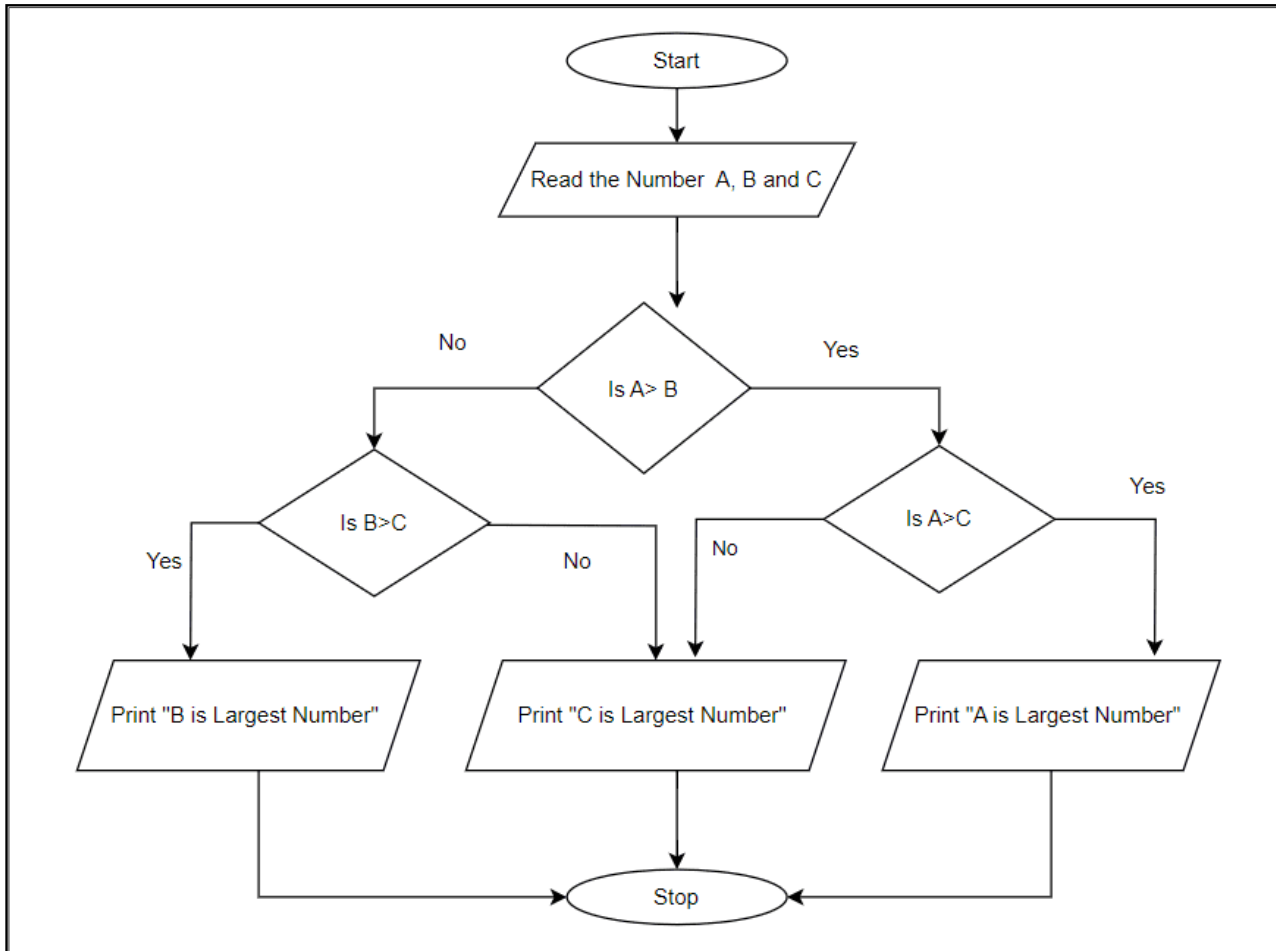
Symbol	Symbol Name	Description
	start/stop Terminal	This symbol is used to represent start and stop of the flow chart.
	Input/output	This symbol is used to represent the input and output of the flow chart.
	Processing	This symbol is used to represent the processing like arithmetic operations, data assignments and movements etc.
	Decision	This symbol is used to check whether condition is true(yes) or false(no).
	Flow Lines(arrows)	This symbol is used to connect the symbols.it indicates the direction of the flow.
	Connector	This symbol is used to connect the flow lines.
	Off-page Connector	This symbol is used to continue the flow chart on Next Page (New page)

Flowchart to find the maximum among three numbers inputted by the user

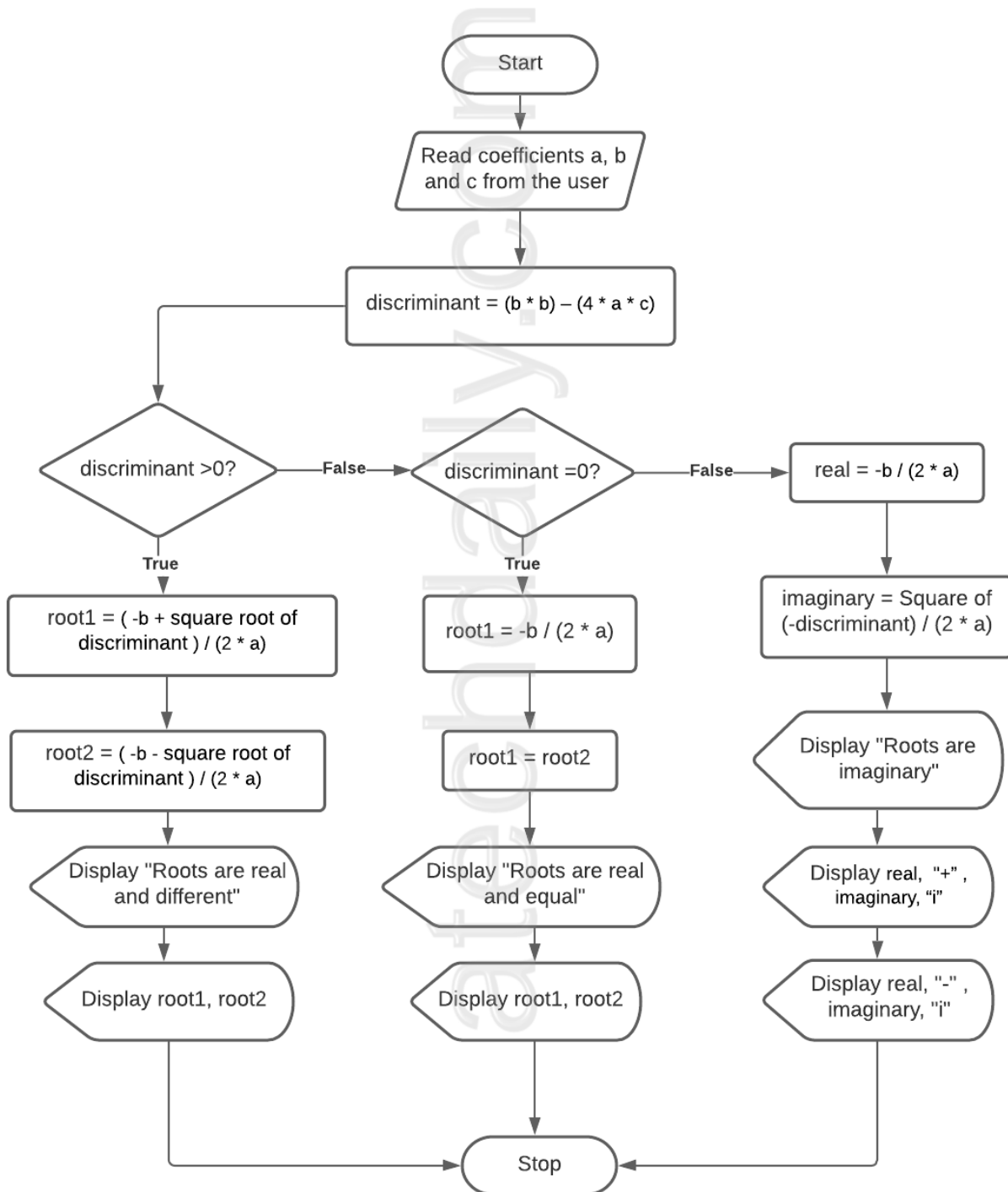
Algorithm:

1. Start
2. Read Three Number A,B and C
3. Check, Is A is greater than B ($A > B$)
4. If Yes, Check, Is A is greater than C ($A > C$)
 - 4.1 If Yes, print "A is Largest Number"
 - 4.2 If No, print "C is Largest Number"

5. If No, Check, Is B is greater than C ($B > C$)
 - 5.1 If Yes, print "B is Largest Number"
 - 5.2 If No, print "C is Largest Number"
6. Stop



Flowchart to find all the roots of a quadratic equation:



Uses of flowchart :

Flowcharts are incredibly useful tools in programming, including C, for several reasons:

1. Graphical Representation of Problem

Flowcharts provide a clear and visual way to represent the logic and flow of a program. This helps in understanding how the program works and in identifying any logical errors or inefficiencies.

2. Divide and simplify the flow of process

They break down complex processes into simpler, more manageable parts. This makes it easier to analyze and debug the program.

3. Communication Tool

Flowcharts serve as an excellent communication tool among team members. They help in explaining the program's structure and logic to others, including those who may not be familiar with programming.

4. Ease of Documentation

Flowcharts act as a form of documentation for the program. They provide a reference that can be used for future maintenance and updates.

5. Make Planning and Design Easy

Before writing any code, flowcharts can be used to plan and design the structure of the program. This helps in organizing thoughts and ensuring that all aspects of the program are considered.

3. Discuss various primitive data types used in C with suitable examples including their required memory size, format specifier and range.

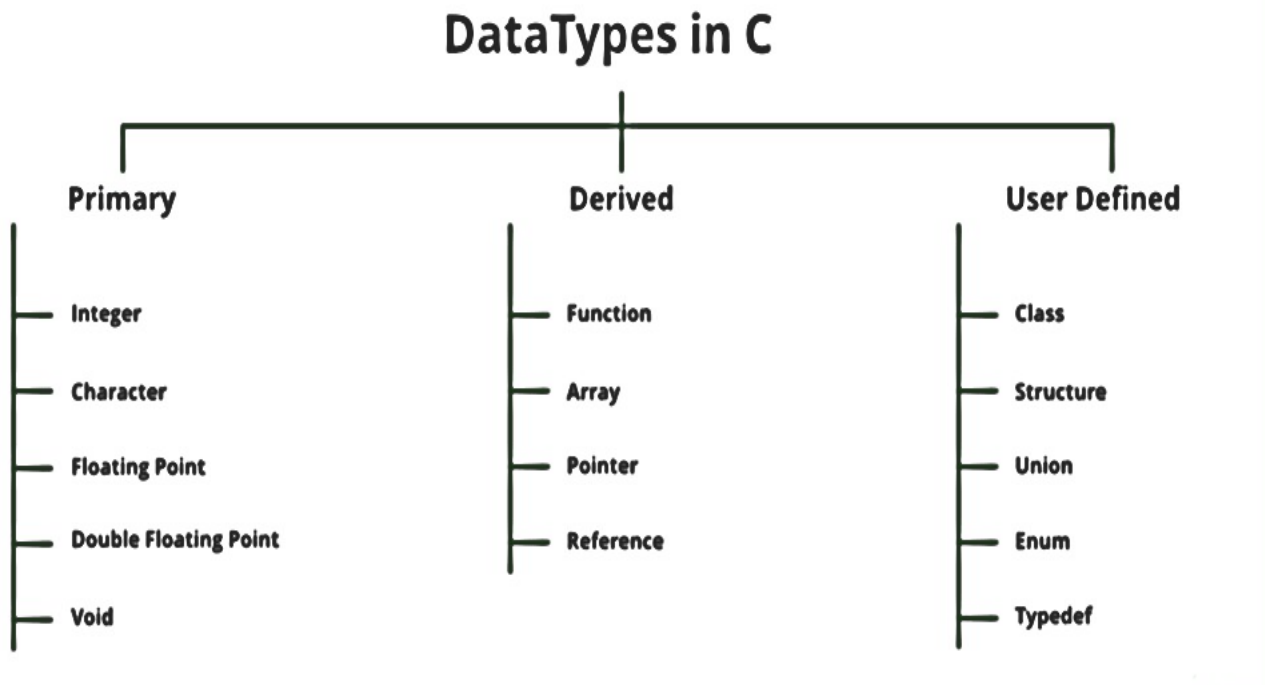
Ans: Data types are the type of data stored in a C program. Data types are used while defining a variable or functions in C. The compiler needs to understand the type of predefined data it will encounter in the program. A data type is an attribute that tells a computer how to interpret the value.

“The term “data type” that describes the kind of value a variable”

The data comes in different forms. Examples include:

- **your name** – a string of characters
- **your age** – usually an integer
- **the amount of money in your pocket**- usually decimal type
- **today’s date** – written in date time format

The data types in C can be classified as follows:



The following are some main primitive data types in C:

➤ **Integer Data Type:**

The integer datatype in C is used to store the integer numbers (any number including positive, negative and zero without decimal part). Octal values, hexadecimal values, and decimal values can be stored in int data type in C.

- **Range:** -2,147,483,648 to 2,147,483,647
- **Size:** 4 bytes
- **Format Specifier:** %d

➤ **Character Data Type:**

Character data type allows its variable to store only a **single character**.

The size of the character is 1 byte. It is the most basic data type in C.

It stores a single character and requires a single byte of memory in almost all compilers.

- **Range:** (-128 to 127) or (0 to 255)
- **Size:** 1 byte
- **Format Specifier:** %c

➤ **Float Data Type:**

In C programming [float data type](#) is used to store floating-point values.

Float in C is used to store decimal and exponential values.

It is used to store decimal numbers (numbers with floating point values) with single precision.

- **Range:** 1.2E-38 to 3.4E+38
- **Size:** 4 bytes

- **Format Specifier:** %f

Range of data types:

Data type	Bytes in Ram	Range of data type
char	1 bytes	-128 to 127
int	2 bytes	-32, 768 to 32,767
float	4 bytes	3.4c-38 to 3.4 c+ 38
double	8 bytes	1.7C – 308 to 1.7c +308

Derived data types in C

- **Array:** An array in C is a collection of multiple values of a similar data type and is stored in a contiguous memory location. An array can consist of chars, integers, doubles, etc.
- Declaration of Array in C: `data_type array_name[array_size];`
- **Pointer Data Type:** The pointer data type is used to store the address of another variable. A pointer can store the address of variables of any data type. Pointers allow users to perform dynamic memory allocation. They also help to pass variables by reference.
- Declaration of pointer : `data_type *pointer_variable_name;`

User-defined data types in C

- **Structure:** It is a data type that can store variables of similar or different data types.
- For example, we can use structures to store information about an employee, such as the employee's name, employee ID, salary, and more. Each employee's record will be represented by an object of the structure. The size of the structure is the sum of the storage size required by each variable.
- The 'struct' keyword defines a structure.
- **Union:** A union is a group of elements with similar or different data types. In a union, the memory location is the same for all the elements.
- Its size will be equal to the **memory required for the largest data type** defined. We use the keyword 'union' to define a union. You can declare many variables. However, just one variable can store the value at a time.

- **Enumerated Data Types:** Enumerated data types are user-defined data types that consist of integer values. They are used to define variables that can only assign certain discrete integer values in the program. They are used to make a program more readable, flexible, and maintainable.
- We use the keyword 'enum' to declare new enumeration types in the C programming language.
Enum syntax: enum flag {const1, const2, const3.....};

b. What do you mean by typecasting? Illustrate implicit type casting and explicit type casting with suitable examples.

Ans: Type casting in C refers to the process of converting a value from one data type to another.

This conversion can happen automatically (implicit casting) or be explicitly specified by the programmer (explicit casting).

There are two types of type casting: implicit and explicit.

a) Implicit Type Casting (Coercion)

b) Explicit Type Casting (Casting)

Implicit Type Casting :-In implicit type casting, the programming language automatically converts data from one type to another if needed.

For example, if you have an **integer** variable and you try to assign it to a **float** variable, the programming language will automatically convert the integer to a float without you having to do anything.

```
#include <stdio.h>

int main() {
    int integer_num = 10;
    float float_num = 3.14;

    // Implicit casting in arithmetic operation
    float result = integer_num + float_num;

    printf("Result of int + float: %f\n", result);

    // Implicit casting in assignment
    int truncated = float_num;

    printf("Truncated float to int: %d\n", truncated);

    return 0;
}
```

```
}
```

Explicit type casting:- It is also known as type conversion or type coercion, occurs when the programmer explicitly converts a value from one data type to another.

Unlike implicit type casting, explicit type casting requires the programmer to specify the desired data type conversion.

```
#include <stdio.h>

int main() {
    int num1 = 17, num2 = 5;
    float result;

    // Without explicit casting
    result = num1 / num2;
    printf("Result without casting: %f\n", result);

    // With explicit casting
    result = (float)num1 / num2;
    printf("Result with casting: %f\n", result);

    // Casting in complex expressions
    int x = 10, y = 3;
    float z = 7.5;

    result = (float)(x + y) / z;
    printf("Complex casting result: %f\n", result);

    return 0;
}
```

Output:

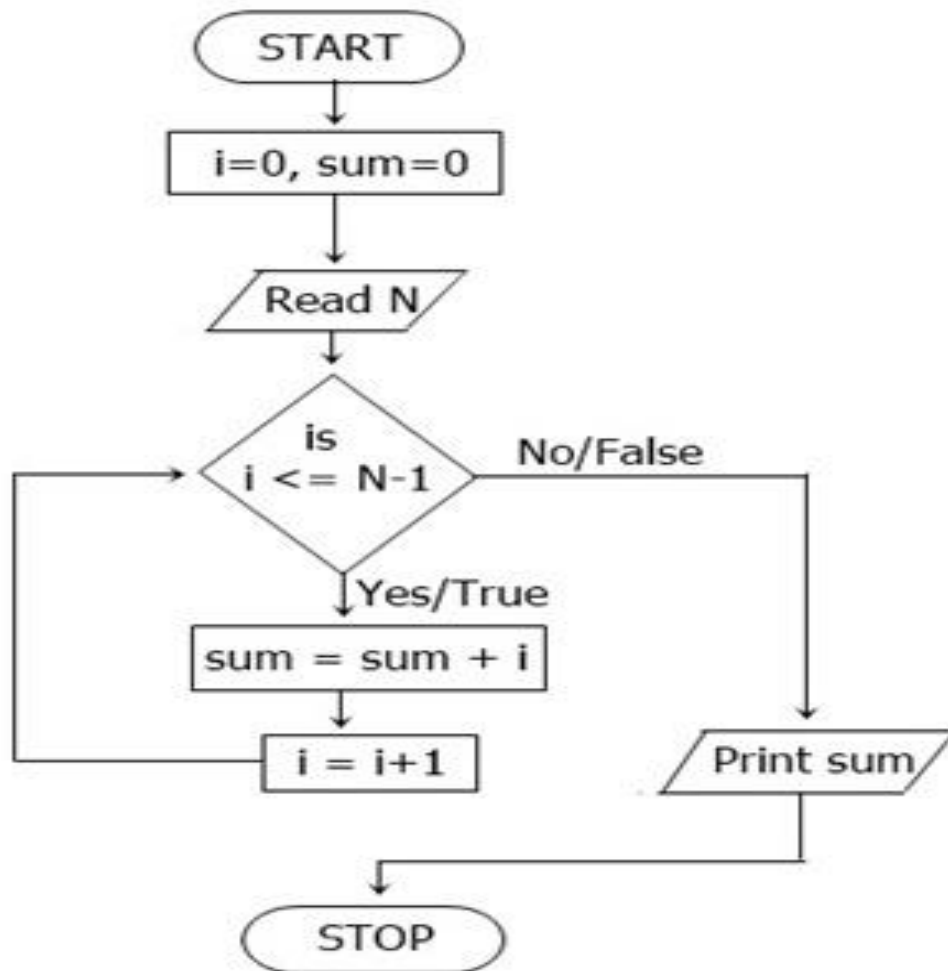
```
Result without casting: 3.000000
Result with casting: 3.400000
Complex casting result: 1.733333
```

In this example:

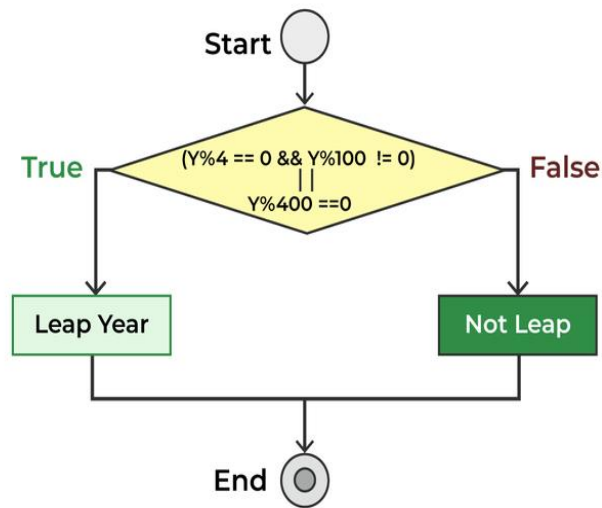
1. Without casting, `num1 / num2` performs integer division, resulting in 3.
2. With casting, `(float)num1 / num2` converts `num1` to a float before division, giving the correct result of 3.4.
3. In the complex expression, `(x + y)` is first evaluated as an integer, then }

4. Draw the flowchart of the following problems:

a) Draw a flow chart to find the sum and average of n integers.



b). Draw a flow chart to find whether the given year is a leap year or not.



5. What do you mean by operator precedence and associativity? Explain all bitwise AND, bit-wise OR and bit-wise XOR operators with suitable example.

Ans: Operator precedence: It dictates the order of evaluation of operators in an expression.

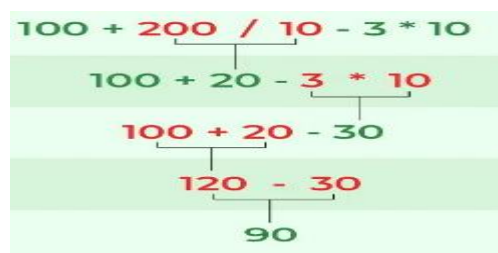
Associativity: It defines the order in which operators of the same precedence are evaluated in an expression. Associativity can be either from left to right or right to left.

In C, each **operator has a fixed priority or precedence** in relation to other operators.

As a result, the operator with **higher precedence is evaluated first** or before the operator with lower precedence.

Operators that appear in the same group have the same precedence. The following table lists operator precedence and associativity.

For example:



(highest to lowest)

<i>Operator</i>	<i>Associativity</i>
()	Left to right
unary: ++ -- ! + - (cast) sizeof	Right to left
* / %	Left to right
+ -	Left to right
< <= > >=	Left to right
== !=	Left to right
&&	Left to right
	Left to right
? :	Right to left
= += -= *= /=	Right to left

Types of Associativity:

- Left-to-right: Most operators, like +, -, or *, follow this rule.
- Right-to-left: Operators like = (assignment) or ? : (conditional) follow this rule.

Bitwise Operators

Bitwise operators perform operations on the **individual bits of numbers**, which are represented in **binary format**.

These operators are commonly used in low-level programming for tasks like masking, shifting, and toggling bits.

Bitwise AND Operator in C (&)

The bitwise AND operator (&) compares each **bit of two operands**.

It sets each bit in the result to 1 if both corresponding bits in the operands are 1; otherwise, it sets the bit to 0.

This operator is commonly used for bit masking, where specific bits need to be checked, set, or cleared.

Syntax:

```
result = operand1 & operand2;
```

```

#include <stdio.h>

int main() {

    int a = 5; // Binary: 0101

    int b = 3; // Binary: 0011

    int result = a & b;

    printf("Result of a & b = %d\n", result);

    return 0;

}

```

Operator Symbol	Operator Name	Description	Example
&	Bitwise AND	Sets each bit to 1 if both bits are 1	a & b
	Bitwise OR	Sets each bit to 1 if at least one of the bits is 1	a b
^	Bitwise XOR	Sets each bit to 1 if only one of the bits is 1	a ^ b
~	Bitwise NOT	Inverts all the bits (1 becomes 0 and 0 becomes 1)	~a
<<	Left Shift	Shifts bits to the left and fills 0 from the right	a << n
>>	Right Shift	Shifts bits to the right and fills 0 from the left (for unsigned)	a >> n

Note: Complete all types of operator same in this manner.

Unit-2

1. How do-while loop is different from a while loop? Give the syntax and working of both the loops.

Ans.

While loop/Entry Controlled	do-while loop/Exit Controlled
Condition is checked first then statement(s) is executed.	Statement(s) is executed at least once, thereafter condition is checked.
It might occur that statement(s) is executed zero times, if the condition is false.	At least once the statement(s) is executed.
No semicolon at the end of the while. while(condition)	Semicolon at the end of the while. while(condition);
If there is a single statement, brackets are not required.	Brackets are always required.
The variable in the condition is initialized before the execution of the loop.	variable may be initialized before or within the loop.
while loop is an entry-controlled loop.	The do-while loop is an exit-controlled loop.
while(condition) { statement(s); }	do { statement(s); } while(condition);
<pre>#include <stdio.h> void main() { int i = 5; while (i < 10) { printf("GFG\n"); i++; } getch(); }</pre>	<pre>#include <stdio.h> void main() { int i = 5; do { printf("GFG\n"); i++; } while (i < 10); getch(); }</pre>

Write a program to print Floyd's Triangle in C

```
#include <stdio.h>
int main()
{
    int rows = 4;
    int n = 1;
    // outer loop to print all rows
    for (int i = 0; i < rows; i++) {
        // inner loop to print alphabet in each row
        for (int j = 0; j <= i; j++) {
            printf("%d ", n++);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

2. Give the syntax and explain the working of the switch-case statement. WAP in C to implement the menu-based basic calculator using the switch case.

OR

This program can be asked like “Give the syntax and explain the working of the switch-case statement. WAP in a modular program in C to implement the menu-based calculator.”

Ans. The switch statement in C is a control flow mechanism that allows a program to make decisions based on the value of a variable.

- It is an **efficient and readable alternative** to a series of if-else statements.
- it **consists of an expression** that is evaluated and compared to a series of case values.

In the switch case statement,

- If a **case value matches** the expression, the corresponding **statements are executed** and the program continues until it encounters a break statement or reaches the end of the switch statement.
- If no case values match the expression, the default statement is executed, if provided.
- The switch statement is often used for making decisions based on well-defined, large sets of values.
- It provides a more concise and maintainable solution than complex nested if-else statements.

Syntax of the Switch Case Statement in C:

```
switch (expression)
{
    case constant-expression:
        statement(s);
        break;
    case constant-expression:
        statement(s);
        break;
    ...
    default:
        statement(s);
}
```

Let's understand the above syntax of the switch case statement in c:

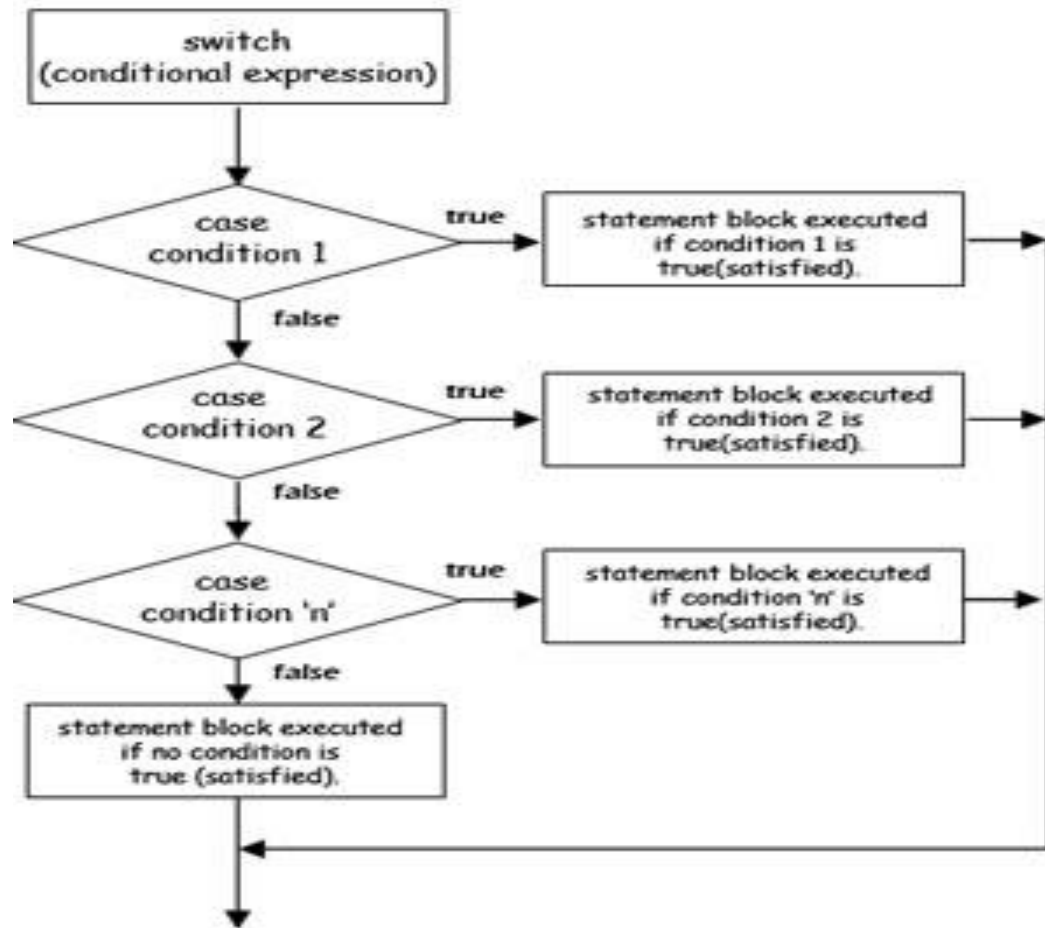
Rules For switch statement:

- The switch keyword is followed by an expression in parentheses.
- The case keyword is followed by a constant expression and a colon.
 - If the value of the switch expression **matches the constant expression** in a case label the statements following that **case label will be executed**.
- **One or more statements** that follow a case label.
- The optional **break statement terminates** the switch statement and transfers control to the statement following the switch statement.
- The optional **default keyword is followed by a colon** and one or more statements.
- The statements following the **default label are executed** if the value of the switch expression **does not match any of the constant expressions** in the case labels.

Advantages of Switch statements against if –else statement

- **Easier to debug** (e.g. setting breakpoints on code vs. a call table, if the debugger has no conditional breakpoint capability)
- **Easier to read** (subjectively)
- **Easier to understand**, and therefore
- **Easier to maintain**
- **Fixed depth**: a sequence of "if else if" statements yields deep nesting, making compilation more difficult (especially in automatically generated code)
- **Faster execution potential**
- **Easier to study and write** the examination for students

Flowchart of the Switch Statement in C:



C program to implement the menu-based basic calculator using the switch case.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    char op;
    float num1, num2, result=0.0;

    /* Print welcome message */
    printf("WELCOME TO SIMPLE CALCULATOR\n");
    printf("-----\n");
    printf("Enter [number 1] [+ - * /] [number 2]\n");

    /* Input two numbers and operator from the user */
    scanf("%f %c %f", &num1, &op, &num2);

    /* Switch the value and perform an action based on operator*/
    switch(op)
```

```

{
    case '+':
        result = num1 + num2;
        break;

    case '-':
        result = num1 - num2;
        break;

    case '*':
        result = num1 * num2;
        break;

    case '/':
        result = num1 / num2;
        break;
    default:
        printf("Invalid operator");
}
/* Prints the result */
printf("%.2f%c %.2f = %.2f", num1, op, num2, result);
getch();
}

```

3. Define function. What are the advantages of using functions? Write a C program to interchange the contents of two variables using a function.

Ans. In C, we can divide a large program into the basic building blocks known as functions.

- The function contains the set of programming statements **enclosed by {}**.
- A function can be called multiple times to provide **reusability and modularity** to the C program.
- In other words, we can say that the **collection of functions creates a program**. The function is also known as **procedure or subroutine** in other programming languages.
-

Advantage of functions in C

There are the following advantages of C functions.

- By using functions, we can **avoid rewriting the same logic/code** again and again in a program.
- We can call C functions **any number of times in** a program and from any place in a program.
- We can track a large C program easily when it is **divided into multiple functions**.
- **Reusability** is the main achievement of C functions.
- However, Function calling is always an overhead in a C program.

C program to interchange the contents of two variables using the function

```
void swap(int *num1, int *num2)
{
    int temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
void main()
{
    int var1, var2;
    printf("Enter Value of var1 ");
    scanf("%d", &var1);
    printf("\nEnter Value of var2 ");
    scanf("%d", &var2);
    swap(&var1, &var2);
    printf("\nAfter Swapping: var1 = %d, var2 = %d", var1, var2);
}
```

NOTE: The above code example is also perfect for explaining “Call By Reference”

What is call by reference: "Call by reference" is a method of passing arguments to a function in a programming language. In languages that support call by reference, when you pass a parameter by reference, you're actually passing a reference (memory address) to the variable rather than the value itself. This means that changes made to the parameter within the function affect the original variable outside the function.

4. Write a C program to check whether a triangle is right-angled or not.

Ans.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int x, y, z;

    // Prompt the user to input the three sides of a triangle
    printf("Input the three sides of a triangle:\n");

    // Read the input values for x, y, and z
    scanf("%d %d %d", &x, &y, &z);

    // Check if the triangle is a right angle triangle
```

```

if ((x*x) + (y*y) == (z*z) || (x*x) + (z*z) == (y*y) || (y*y) + (z*z) == (x*x))
{
    // If the condition is true, print that it's a right angle triangle
    printf("It is a right angle triangle!\n");
}
else
{
    // If the condition is false, print that it's not a right angle triangle
    printf("It is not a right angle triangle!\n");
}

getch();
}

```

NOTE: Always use Pythagoras approach to write this program as it gives the desired clarity with mathematical alignment of program

5.WAP in C to print the following pattern:

*	1	A
* *	1 2	A B
* * *	1 2 3	A B C
* * * *	1 2 3 4	A B C D
* * * * *	1 2 3 4 5	A B C D E

// c program to print left half pyramid pattern of star
#include <stdio.h>

```

int main()
{
    int rows = 5;

    // first loop is for printing the rows
    for (int i = 0; i < rows; i++) {

        // loop for printing leading whitespaces
        for (int j = 0; j < 2 * (rows - i) - 1; j++) {

```

```
        printf(" ");
    }

    // loop for printing * character
    for (int k = 0; k <= i; k++) {
        printf("* ");
    }
    printf("\n");
}
return 0;
}
```

Unit 3

1. Illustrate different ways to input a string in C language. WAP to count the number of characters and words inputted in a string.

Ans. Different ways to input a string in C language are:

- a. **scanf():** In C, we can use scanf() to take a string input in C without spaces. Like other data types, we have an access specifier (also known as format specifier) to take input as a string in C as well. The access specifier for string is %s.

The syntax for using scanf() function in C :

```
scanf("%s", char *s);
```

Here, s is the pointer which points to the array of characters where the input taken as a string will be stored.

Note that in the syntax, we don't use the & symbol with s. This is because & is used to access the address of the variable, but as C doesn't have a string data type, s already stores the address of the character array where the string is stored.

Let us look at an example to understand the working of scanf() in C.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // array to store string taken as input
```

```
    char color[20];
```

```
    // take user input
```

```
    printf("Enter your favourite color: ");
```

```
    scanf("%s", color);
```

```
    // printing the input value
```

```
    printf("Your favourite color is: %s.", color);
```

```
    return 0;
```

```
}
```

- b. **gets():** This is a C standard library function. gets() takes a complete line as input and stores it in the string provided as a parameter. The working of gets() is such that it keeps reading the input stream until it encounters a newline character: \n. Hence, even if the string input in C has spaces in it, all of them get included as input until \n doesn't occur.

The syntax for gets() in C is:

```
char* gets(char* s)
```

Here, s is the pointer which points to the array of characters where the input taken as a string will be stored and returned. This function will return the string in s when it finishes successfully.

```
#include <stdio.h>
int main()
{
    // array to store string taken as input
    char sentence[30];

    // take user input
    printf("Enter any sentence: ");
    // use the gets method
    gets(sentence);
    // printing the input value
    printf("You entered: %s.", sentence);
    return 0;
}
```

gets() seems to be a great method to take string input in C with spaces. But gets() doesn't care about the size of the character array passed to it. This means, in the above example, if the user input would be more than 30 characters long, the gets() function would still try to store it in the sentence[] array, but as there is not much available space, this would lead to buffer overflow. Because of this limitation, using the gets() function is not preferable.

c. fgets()

To overcome the above-listed limitation, we can use fgets() function. The fgets() function is similar to gets(), but we can also specify a maximum size for the string which will be taken as a string input in C from the user. This means that it reads either the complete line as a string and stores it in the array if the size is less than what is specified as a parameter (say: sz), else it reads only sz characters provided by the user so that no buffer overflow occurs.

The syntax for fgets() in C is:

```
char* fgets(char* s, int sz, FILE* stream)
```

Here, s is the pointer which points to the array of characters where the input taken as a string will be stored and returned. The sz variable specifies the maximum allowed size for the user input and stream is used to denote the stream from which the input is to be taken (usually stdin, but it can also be a file).

```
#include <stdio.h>
int main()
{
    // array to store string taken as input
    char sentence[20];

    // take user input
```



```

printf("Enter any sentence: ");

// use the fgets method specifying the size of the array as the max limit
fgets(sentence, 20, stdin);

// printing the input value
printf("You entered: %s.", sentence);
return 0;
}

```

Program to count the number of characters and words inputted in a string.

```

#include <stdio.h>
#include <string.h>
#include <conio.h>
void main()
{
    char str[50];
    int i=0, word=0, chr=0;
    printf("\nEnter Your String: ");
    gets(str);
    while (str[i] != '\0')
    {
        if (str[i] == ' ')
        {
            word++;
            chr++;
        }
        else
            chr++;
        i++;
    }
    printf("\nNumber of characters: %d", chr);
    printf("\nNumber of words: %d", word+1);
    getch();
}

```

2. Define a pointer. How do you declare and initialize a pointer? WAP to find the sum of an integer array elements using pointers.

Ans. A pointer is defined as a derived data type that can store the address of other C variables or a memory location. We can access and manipulate the data stored in that memory location using pointers.

Pointers can be very useful in some cases, like:

They make accessing array elements easier.

We can implement linked lists, trees, and graphs using pointers.

We use pointers for accessing dynamically allocated memory.

We can return more than one value from a function using pointers.

Syntax of C Pointers

The syntax of pointers is similar to the variable declaration in C, but we use the (*) dereferencing operator in the pointer declaration.

```
datatype * ptr;
```

where

- ptr is the name of the pointer.
- datatype is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

Pointer Declaration

In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the (*) dereference operator before its name.

Example

```
int *ptr;
```

The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

Pointer Initialization

Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use the (&) addressof operator to get the memory address of a variable and then store it in the pointer variable.

Example

```
int var = 10;
```

```
int * ptr;
```

```
ptr = &var;
```

We can also declare and initialize the pointer in a single step. This method is called pointer definition as the pointer is declared and initialized at the same time.

Example

```
int *ptr = &var;
```

Program to find the sum of an integer array elements using pointers.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int array[5];
```

```
    int i, sum=0;
```

```
    int *ptr;
```

```
    printf("\nEnter array elements (5 integer values):");
```

```
    for(i=0;i<5;i++)
```

```
        scanf("%d",&array[i]);
```

```

/* array is equal to base address
 * array = &array[0] */
ptr = array;

for(i=0;i<5;i++)
{
    /*ptr refers to the value at address
    sum = sum + *ptr;
    ptr++;
}
printf("\nThe sum is: %d",sum);
getch();
}

```

3. **Declare and initialize a 2-dimensional array during compile time. Write a program in C to search a number in a given array of numbers.**

Ans. A two-dimensional array or 2D array in C is the simplest form of the multidimensional array. We can visualize a two-dimensional array as an array of one-dimensional arrays arranged one over another forming a table with 'x' rows and 'y' columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1).

Declaration of Two-Dimensional Array in C

Syntax:

```
data_type array_name[x][y];
```

where,

- data_type: Type of data to be stored in each element.
- array_name: name of the array
- x: Number of rows.
- y: Number of columns.

We can declare a two-dimensional integer array say 'x' with 10 rows and 20 columns as:

Example:

```
int x[10][20];
```

Initialization of Two-Dimensional Arrays in C

A 2D array can be initialized as follows:

Initialization of 2D array using Initializer List: We can initialize a 2D array in C by using an initializer list as shown in the example below.

First Method:

```
int x[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

The above array has 3 rows and 4 columns. The elements in the braces from left to right are stored in the table also from left to right. The elements will be filled in the

array in order: the first 4 elements from the left will be filled in the first row, the next 4 elements in the second row, and so on.

Second Method (better):

```
int x[3][4] = {{0,1,2,3}, {4,5,6,7}, {8,9,10,11}};
```

This type of initialization makes use of nested braces. Each set of inner braces represents one row. In the above example, there is a total of three rows so there are three sets of inner braces. The advantage of this method is that it is easier to understand.

Program in C to search a number in a given array of numbers.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int arr[10],i,n,ele;
```

```
    printf("Enter array size: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter array elements: ");
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    printf("Enter element to search: ");
```

```
    scanf("%d", &ele);
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        if(arr[i]==ele)
```

```
        {
```

```
            printf("%d found at position %d", ele, i+1);
```

```
            return 0;
```

```
        }
```

```
    }
```

```
        printf("Element not found");
```

```
}
```

4. Write a program in C to read n number of values in an array and display them in reverse order.

Ans.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
// Main function
```

```
void main()
```

```
{
```

```

int i, n, a[100];
// Display a message to the user about the program's purpose
printf("\n\nRead n number of values in an array and display it in reverse order:\n");
printf("-----\n");
// Prompt the user to input the number of elements to store in the array
printf("Input the number of elements to store in the array :");
scanf("%d", &n);
// Prompt the user to input n elements into the array
printf("Input %d number of elements in the array :\n", n);
for (i = 0; i < n; i++)
{
    printf("element - %d : ", i);
    scanf("%d", &a[i]); // Read the input and store it in the array
}
// Display the values stored in the array
printf("\nThe values stored in the array are : \n");
for (i = 0; i < n; i++)
{
    printf("%5d", a[i]); // Print each element in the array
}
// Display the values stored in the array in reverse order
printf("\n\nThe values stored in the array in reverse are :\n");
for (i = n - 1; i >= 0; i--)
{
    printf("%5d", a[i]); // Print each element in reverse order
}
printf("\n\n");
getch();
}

```

5. Write a program in C to find the sum of rows and columns of a matrix.

Ans.

```

#include <stdio.h>
#include <conio.h>
void main() {
    int i, j, k, arr1[10][10], rsum[10], csum[10], n;

    // Display the purpose of the program
    printf("\n\nFind the sum of rows and columns of a Matrix:\n");
    printf("-----\n");

    // Input the size of the square matrix

```

```

printf("Input the size of the square matrix : ");
scanf("%d", &n);

// Input elements into the matrix
printf("Input elements in the matrix :\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("element - [%d],[%d] : ", i, j);
        scanf("%d", &arr1[i][j]);
    }
}

// Display the matrix
printf("The matrix is :\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("% 4d", arr1[i][j]);
    }
    printf("\n");
}

// Calculate the sum of rows
for (i = 0; i < n; i++) {
    rsum[i] = 0;
    for (j = 0; j < n; j++) {
        rsum[i] = rsum[i] + arr1[i][j];
    }
}

// Calculate the sum of columns
for (i = 0; i < n; i++) {
    csum[i] = 0;
    for (j = 0; j < n; j++) {
        csum[i] = csum[i] + arr1[j][i];
    }
}

// Display the matrix with row sums
printf("The sum of rows and columns of the matrix is :\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("% 4d", arr1[i][j]);
    }
}

```

```

        printf("% 8d", rsum[i]); // Print the sum of each row
        printf("\n");
    }

    // Display the column sums
    printf("\n");
    for (j = 0; j < n; j++) {
        printf("% 4d", csum[j]); // Print the sum of each column
    }
    printf("\n\n");

    getch();
}

```

Unit 4

1. Define the scope and lifetime of a variable. Explain in detail all storage classes supported in C language with reference to scope and lifetime.

Ans. Scope is defined as the availability of a variable inside a program, scope is the region of code in which a variable is available to use.

There are four types of scope:

file scope,

block scope,

function scope and

prototype scope.

The lifetime of a variable is the time for which the variable is taking up a valid space in the system's memory, it is of three types:

static lifetime,

automatic lifetime and

dynamic lifetime.

Storage Class	Storage	Default Value	Scope	Life
auto	Main Memory (RAM)	Garbage	Local to block in which the variable is declared	Within the function in which it is declared.

register	CPU Register	Garbage	Local to block in which the variable is declared	Within the function in which it is declared.
static	Main Memory (RAM)	Zero (0)	Both local and Global	Variable retains its value between different function call.
extern	Main Memory (RAM)	Zero (0)	Global	Throughout the program execution.

2. How is structure different from union? WAP to store 50 players' details such as Player Name, Country Name and Batting average. Using the Database display the details of the players who are playing for a given country.

Ans. Structures in C is a user-defined data type available in C that allows to combining of data items of different kinds. Structures are used to represent a record.

```
struct [structure name]
{
    member definition;
    member definition;
    ...
    member definition;
};
```

(OR)

```
struct [structure name]
{
    member definition;
    member definition;
    ...
    member definition;
}structure variable declaration;
```

Union in C is a special data type available in C that allows storing different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purposes.

```
union [union name]
{
```



```

    member definition;
    member definition;
    ...
    member definition;
};

```

(OR)

```

union [union name]
{
    member definition;
    member definition;
    ...
    member definition;
}union variable declaration;

```

Differences between Structure and Union are as shown below in tabular format as shown below as follows:

Program using structures:

```

#include <stdio.h>
#include <string.h>
#include <conio.h>
struct cricket
{
    char player_name[20];
    char country_name[20];
    float batting_avg;
}p[50],t;

void main()
{
    int i=0,j=0,n=50;

    for(i=0;i<n;i++)
    {
        printf("\n Enter Player Name : ");
        scanf("%s",p[i].player_name);
        printf("\n Enter Team Name : ");
        scanf("%s",p[i].team_name);
        printf("\n Enter Batting Average : ");
        scanf("%f",&p[i].batting_avg);
    }
}

```

```

}

//Sorting of Data based on Country
for(i=0;i<n-1;i++)
{
    for(j=i;j<n;j++)
    {
        if(strcmp(p[i].country_name,p[j].country_name)>0)
        {
            t=p[i];
            p[i]=p[j];
            p[j]=t;
        }
    }
}

j=0;
for(i=0;i<n;i++)
{
    if(strcmp(p[i].country_name,p[j].country_name)!=0 || i==0)
    {
        printf("\n Country Name: %s",p[i].country_name);
        j=i;
    }
    printf("\n Player Name    = %s",p[i].player_name);
    printf("\n Batting Average = %f",p[i].batting_avg);
}
getch();
}

```

NOTE: This question can be asked for 50 Employees or 50 Students or any other form of complex entity. The approach of answering will be same, just need to use attributes given in the question for the entity.

3. WAP in C to Add Two Distances (in an inch-feet system) using Structures (12 inches is 1 foot)

Ans.

```

#include <stdio.h>
#include <conio.h>

```

```

struct Distance {
    int feet;
    float inch;
}

```

```

} d1, d2, result;

void main() {
    // take first distance input
    printf("Enter 1st distance\n");
    printf("Enter feet: ");
    scanf("%d", &d1.feet);
    printf("Enter inch: ");
    scanf("%f", &d1.inch);

    // take second distance input
    printf("\nEnter 2nd distance\n");
    printf("Enter feet: ");
    scanf("%d", &d2.feet);
    printf("Enter inch: ");
    scanf("%f", &d2.inch);

    // adding distances
    result.feet = d1.feet + d2.feet;
    result.inch = d1.inch + d2.inch;

    // convert inches to feet if greater than 12
    while (result.inch >= 12.0) {
        result.inch = result.inch - 12.0;
        ++result.feet;
    }
    printf("\nSum of distances = %d\'-%.1f'", result.feet, result.inch);
    getch();
}

```

4. Explain the concept of unions in C with the help of examples.

Ans. In C programming language, there is a special user-defined data type called union that allows variables of different data types to be stored in the same memory location. A union can be defined with multiple members, but only one of those members can contain a value at a given time.

Defining a Union in C

The syntax of the union statement in C is given below:

```

union union_tag
{
    datatype member1;
    datatype member2;
    ...
}

```

```
    datatype memberN;  
} [one or more union variables];
```

5. Write a C program to accept details of 'n' employee (eno, ename, salary) and display the details of employee having highest salary. Use array of structure.

Ans.

```
#include<stdio.h>  
#include<conio.h>  
struct employee  
{  
    int eno;  
    char ename[20];  
    int salary;  
}emp[10];  
void main()  
{  
    int i,high,n;  
    printf("/*How many employee info\nyou want to accept : ");  
    printf("Enter Limit: ");  
    scanf("%d",&n);  
    printf("-----\n");  
    printf("Enter details for %d employees:",n);  
    printf("\n-----\n");  
    for(i=0;i<n;i++)  
    {  
        printf("Employee Number: ");  
        scanf("%d",&emp[i].eno);  
        printf("Name      : ");  
        scanf("%s",emp[i].ename);  
        printf("Salary    : ");  
        scanf("\n %d",&emp[i].salary);  
        printf("-----\n");  
    }  
    high=emp[0].salary;  
    for(i=0;i<n;i++)  
    {  
        if(emp[i].salary>high)  
            high=emp[i].salary;  
    }  
    printf("Highest salary employee details:");  
    printf("\n-----\n");  
    printf("EMPNO  NAME  SALARY\n");
```

```

for(i=0;i<n;i++)
{
    if(emp[i].salary==high)
        printf("\n %d\t%s\t%d",emp[i].eno,emp[i].ename,emp[i].salary);
}
getch();
}

```

Unit 5

1. What are the drawbacks of static memory allocation? Explain the functions available in C language to allocate the memory dynamically.

Ans. Static Memory allocation

Static memory allocation is also known as Compile-time memory allocation because the memory is allocated during compile time. In this type of memory allocation, the memory that the program can use is fixed i.e. we can not allocate or deallocate memory during the program's execution. In many applications, it is not possible to predict how much memory will be needed by the program at run time.

Disadvantages of Static Memory allocation

- This allocation method leads to memory wastage.
- Memory cannot be changed while executing a program.
- Exact memory requirements must be known.
- If memory is not required, it cannot be freed.

Dynamic Memory allocation

Dynamic memory allocation is also known as Runtime memory allocation because the memory is allocated during runtime or program execution. The allocation and release of the memory space can be done using the library functions of `stdlib.h` header file. These functions allocate memory from a memory area called heap and deallocate this memory whenever not required so that it can be used for some other purpose.

Properties of Dynamic Memory allocation

- Memory is allocated at runtime.
- Memory can be allocated and released at any time.
- Heap memory is used here.
- Dynamic memory allocation is slow.
- It is more efficient as compared to Static memory allocation
- The allocation process is simple is complicated.
- Memory can be resized dynamically or reused.

The library functions of the `stdlib.h` header file, which helps to allocate memory dynamically are:

- `malloc()`
- `calloc()`
- `realloc()`
- `free()`

malloc()

This function is used to allocate the single block of requested memory. On Success, the malloc() function returns a pointer to the first byte of the allocated memory. The malloc() function gives a NULL output when the memory is not enough.

Syntax:

```
ptr=(data_type *)malloc(specified_size)
```

Here in the above syntax, ptr is a pointer of type data_type, and specified_size is the size in bytes required to allocate. The cast (data_type *) typecast the pointer returned by the malloc() function.

Example

```
ptr = (int*)malloc(100);
```

In the above example, the statement allocates 100 bytes of memory space, and the pointer variable ptr stores the address of the first byte in the memory.

calloc()

The calloc() function is used to allocate the memory as a number of elements of a given size. The calloc() function is similar to malloc() function. The only difference is that it takes two argument values. The first argument specifies the number of data items for which space is required, and the second argument specifies the size of each data item.

Syntax:

```
ptr=(data_type *)calloc(n, specified_size)
```

Here in the above syntax, ptr is a pointer of type data_type, n is the number of data items, and specified_size is the size in bytes required to allocate. The cast (data_type *) typecast the pointer returned by the malloc() function.

Example

```
ptr = (int*)calloc(4, sizeof(int));
```

In the above example, the statement allocates four blocks of memory, each block contains 4 bytes, and the starting address is stored in the pointer variable ptr.

realloc()

The realloc() function is used to increase the memory allocated by malloc() or calloc() function. This function alters the size of the memory block without losing the old data. The realloc() function takes two argument values. The first argument is a pointer variable to the block of memory that was previously allocated by calloc() or malloc(), and the second argument is the new size for that memory block.

Syntax:

```
realloc(pointer_variable, n);
```

Here in the above syntax, pointer_variable is a pointer to the block of memory that was previously allocated by malloc() or calloc() function, and n is the new size for that memory block.

Example

```
int *ptr;
```

```
ptr = (int*)malloc(100);
```

```
ptr = (int*)realloc(ptr, 400);
```

Here in the above example, we increased memory size from 100 bytes to 400 bytes.

free()

When memory is allocated dynamically by the malloc() and calloc() function, it should always be released when it is no longer required. Otherwise, it will consume memory until the program exit. The free() function is used to release this allocated memory space.

Syntax:

```
free(pointer_variable);
```

Here in the above syntax, the memory block pointed by the pointer 'pointer_variable' would be released back to the system.

2. Explain the following graphics functions supported by C language:

i.initgraph()

ii.rectangle()

iii.outtextxy()

iv.line()

v.setbkcolor()

Ans. **initgraph() :**

It initializes the graphics system.

To start the graphics system, u must call this function.

Syntax :

```
initgraph(int far *graphdriver,int far *graphmode,char far pathtodriver);
```

This function initializes the graphics system by loading a graphics driver from disk then putting the system into graphics mode.

It also resets all graphics settings(color,current position) to their defaults.

rectangle() :

This function is used to draw rectangle. It draws the rectangle in the current line style, thickness and drawing color.

Syntax :

```
rectangle(int left,int top,int right,int bottom);
```

(left,top) is the upper left corner of the rectangle and (right,bottom) is the bottom right corner of the rectangle.

outtextxy() :

It displays a string at the specified position.

Syntax :

```
outtextxy(int x,int y,textstring);
```

line() :

It draws a line between two specific points.

Syntax :

line(int x1,int y1,int x2,int y2);

Setbkcolor():

Purpose:-Setbkcolor function is used to set background color of the screen.

Syntax:-setbkcolor(COLOR);

Example:-setbkcolor(RED);

3. Write a program in C to count the number of words and characters in a file.

Test Data :

Input the file name to be opened: test.txt

Expected Output:

The content of the file test.txt are :

test line 1

test line 2

test line 3

test line 4

**The number of words in the file test.txt are :
12**

The number of characters in the file test.txt are : 36

Ans.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    FILE *fptr;
```

```
    char ch;
```

```
    int wrd=1,charctr=1;
```

```
    char fname[20];
```

```
    printf("\n\n Count the number of words and characters in a file :\n");
```

```
        printf("-----\n");
```

```
        printf(" Input the filename to be opened : ");
```

```
        scanf("%s",fname);
```

```
    fptr=fopen(fname,"r");
```

```
    if(fptr==NULL)
```

```
    {
```

```
        printf(" File does not exist or can not be opened.");
```

```
    }
```



```

else
{
    ch=fgetc(fptr);
    printf(" The content of the file %s are : ",fname);
    while(ch!=EOF)
    {
        printf("%c",ch);
        if(ch==' '||ch=='\n')
        {
            wrd++;
        }
        else
        {
            charctr++;
        }
        ch=fgetc(fptr);
    }
    printf("\n The number of words in the file %s are : %d\n",fname,wrds-1);
    printf(" The number of characters in the file %s are : %d\n\n",fname,charctr-1);
}
fclose(fptr);
getch();
}

```

4. Write a Program to draw basic graphics construction like line, circle, arc, ellipse and rectangle.

Ans.

```

#include<graphics.h>
#include<conio.h>
void main()
{
    intgd=DETECT,gm;
    initgraph (&gd,&gm,"c:\\tc\\bgi");
    setbkcolor(GREEN);
    printf("\t\t\t\n\nLINE");
    line(50,40,190,40);
    printf("\t\t\t\n\n\nRECTANGLE");
    rectangle(125,115,215,165);
    printf("\t\t\t\n\n\n\n\nARC");
    arc(120,200,180,0,30);
    printf("\t\t\t\n\n\nCIRCLE");
}

```

```

circle(120,270,30);
printf("\t\n\n\nELLIPSE");
ellipse(120,350,0,360,30,20);
getch();
}

```

5. What are the major differences between static and dynamic memory allocation in C?

Ans. Memory Allocation: Memory allocation is a process by which computer programs and services are assigned with physical or virtual memory space. The memory allocation is done either before or at the time of program execution. There are two types of memory allocations:

Compile-time or Static Memory Allocation

Run-time or Dynamic Memory Allocation

Static Memory Allocation: Static Memory is allocated for declared variables by the compiler. The address can be found using the address of operator and can be assigned to a pointer. The memory is allocated during compile time.

Dynamic Memory Allocation: Memory allocation done at the time of execution(run time) is known as dynamic memory allocation. Functions `calloc()` and `malloc()` support allocating dynamic memory. In the Dynamic allocation of memory space is allocated by using these functions when the value is returned by functions and assigned to pointer variables.

Tabular Difference Between Static and Dynamic Memory Allocation in C:

S.No	Static Memory Allocation	Dynamic Memory Allocation
1	In the static memory allocation, variables get allocated permanently, till the program executes or function call finishes.	In the Dynamic memory allocation, variables get allocated only if your program unit gets active.
2	Static Memory Allocation is done before program execution.	Dynamic Memory Allocation is done during program execution.
3	It uses stack for managing the static allocation of memory	It uses heap for managing the dynamic allocation of memory
4	It is less efficient	It is more efficient

5	In Static Memory Allocation, there is no memory re-usability	In Dynamic Memory Allocation, there is memory re-usability and memory can be freed when not required
6	In static memory allocation, once the memory is allocated, the memory size can not change.	In dynamic memory allocation, when memory is allocated the memory size can be changed.
7	In this memory allocation scheme, we cannot reuse the unused memory.	This allows reusing the memory. The user can allocate more memory when required. Also, the user can release the memory when the user needs it.
8	In this memory allocation scheme, execution is faster than dynamic memory allocation.	In this memory allocation scheme, execution is slower than static memory allocation.
9	In this memory is allocated at compile time.	In this memory is allocated at run time.
10	In this allocated memory remains from start to end of the program.	In this allocated memory can be released at any time during the program.
11	Example: This static memory allocation is generally used for array.	Example: This dynamic memory allocation is generally used for linked list.